

PART 6: EXECUTION & DELIVERY EXCELLENCE

If strategy is the brain and discovery is the eyes, execution is the hands. A PM with brilliant strategy who cannot ship is worse than useless — they are confidence-eroding. Conversely, a PM who reliably ships meaningful outcomes earns trust that compounds for their entire career.

This chapter teaches the craft of converting decisions into shipped products. It is highly practical. By the end, you should be able to write a PRD that an engineering team would happily build from.

6.1 What Execution Actually Is

Execution is the discipline of moving from a validated decision to working software in users' hands, on time, at quality, with the right outcome measured. Most failures are not strategic — they are executional. The strategy was right; the team got bogged down, the spec was unclear, the launch was botched, the metrics were never wired up, the rollout was ragged.

Three principles underpin great execution:

1. **Clarity beats speed.** The biggest cause of delay is ambiguity. A team that re-discusses scope every standup loses more time than they'd lose to writing a thorough spec upfront.
2. **Trust beats process.** Light, sensible process backed by trusted relationships outperforms heavy process compensating for distrust. A team where the PM and tech lead respect each other can run circles around a team buried in JIRA discipline.
3. **Outcomes beat output.** Shipping 30 features that no one uses is failure, no matter how well-orchestrated. Shipping 3 features that move metrics is success. PMs are paid for outcomes, not output.

6.2 The Product Requirements Document (PRD) — Comprehensive

The PRD is the PM's primary written artifact. It is where ambiguity goes to die. A great PRD aligns engineering, design, QA, marketing, sales, support, and leadership on what's being built and why. A bad PRD wastes weeks.

What a PRD Is and Isn't

A PRD is:

- A clear articulation of the problem and the user.
- A definition of success and how it'll be measured.
- A description of the experience and the requirements.
- A documentation of decisions and trade-offs made along the way.
- A reference document the team comes back to during build.

A PRD is *not*:

- A design spec. (Wireframes and mockups live in Figma.)
- A technical implementation document. (That's the engineering design doc.)
- A project plan. (That's the timeline / Gantt / milestones.)
- A pitch. (Save the marketing copy for launch comms.)

A PRD answers *what* and *why*. The engineering design doc answers *how*. Keeping these separate makes both better.

The Modern PRD Structure

Different companies have different templates. Here's a battle-tested structure that works across contexts. Adapt to your team.

1. TL;DR (3–5 sentences)
2. Background & Context
3. Problem Statement
4. Goals & Non-Goals
5. Target Users / Personas
6. Success Metrics
7. User Stories / Use Cases
8. Functional Requirements
9. UX & Design (links to Figma)
10. Edge Cases & Error States
11. Technical Considerations / Dependencies
12. Risks & Open Questions
13. Launch Plan
14. Out of Scope
15. References & Appendix

We'll walk through each section.

Section 1: TL;DR

The first 5 sentences anyone reads. If they read nothing else, they should know what we're building, for whom, and why.

Bad TL;DR:

"We are building a feature to improve user experience by adding capabilities for better engagement with our content."

This is fluff. Says nothing.

Good TL;DR:

"We are adding offline reading to our podcast app. Currently, ~22% of weekly active users (WAU) report frustration with playback dropping during commutes; this leads to estimated 8% churn in their second month. Offline reading lets users download episodes for listening without connection. We expect this to reduce M2 churn by 3 percentage points and lift Daily Listening Time by 6%, validated by a 4-week A/B test."

Specific. Quantified. Hypothesis stated. Now anyone reading the rest of the doc has context.

Section 2: Background & Context

A short narrative on how we got here. Include:

- The strategic theme this fits into.
- Recent learnings from research or analytics that motivated this.
- Why now (and not 6 months ago, or 6 months from now).

Keep this to ~3 paragraphs. Cite source documents (interview synthesis, dashboard, prior PRD) so readers can dig deeper if they want.

Section 3: Problem Statement

Use the structure from Part 3.13:

[User segment] currently experiences [pain] when [situation]. This causes [consequence], which we know because [evidence]. They have tried [alternatives] but those fail because [limitations]. Solving this would mean [success], measured by [metric].

This section is the heart of the PRD. If the team disagrees on this section, do not move forward. Engineering will build the wrong thing. Design will solve the wrong problem.

Section 4: Goals & Non-Goals

A two-column section that does more for clarity than almost any other.

Goals: Specific, measurable outcomes this initiative is meant to achieve.

Non-Goals: Specific things that this initiative is *not* trying to achieve, even though stakeholders might expect them.

Example for the offline-reading feature:

Goals	Non-Goals
Allow users to download specific episodes for offline playback	Allow users to download full subscriptions / all episodes automatically
Reduce frustration related to connection drops	Solve poor audio quality on weak connections
Improve M2 retention by 3 percentage points	Improve M0 acquisition or M1 activation
Work on iOS and Android	Work on web (separate initiative)
Support up to 10 simultaneous downloads per user	Support unlimited downloads (cost concerns)

Non-goals are gold. They prevent scope creep, manage stakeholder expectations, and force you to commit to what you're not doing. The best PRDs have rich Non-Goals sections.

Section 5: Target Users

Don't write "all users." Specify:

- Which persona(s)?
- Which segment within those personas?
- Which platforms?
- Which geographies?
- Which user state (new, returning, power, churned)?

Example:

- *Primary*: Indian podcast listeners aged 25–45 who use the app on commutes (estimated 2.1M WAU based on event data).
- *Secondary*: International users on flaky connections (rural US, parts of LATAM).
- *Out of focus*: Casual listeners (less than 1 session/week) — they don't have the pain.

Section 6: Success Metrics

This is where many PRDs go fatally vague. Be specific.

Three tiers of metrics:

1. **Primary metric**: The single most important measure of success. Tied to the hypothesis. (E.g., "Lift M2 retention from 38% to 41%, measured by cohort analysis 8 weeks post-launch.")
2. **Secondary metrics**: What else we expect to move. (E.g., "Daily listening time per user up by 6%; in-app session length up by 8%.")
3. **Guardrail metrics**: What must *not* get worse. (E.g., "App crash rate must not increase by more than 0.1%; data usage complaints must not exceed 5/week; storage-out-of-space support tickets stay under baseline.")

For each metric, state:

- Current baseline
- Target
- How it'll be measured (which dashboard, what query)
- When we'll check (post-launch + 4 weeks, etc.)
- What we'll do if we don't hit it

This last point is crucial. Most teams ship features and never come back. State explicitly: "If we don't see lift within 4 weeks, we will [iterate / kill / re-evaluate]."

Section 7: User Stories / Use Cases

User stories describe specific journeys through the feature. Use the format:

As a [user type], when I [trigger/situation], I want to [action], so I can [outcome].

Examples:

- "As a daily commuter, when I'm about to leave home with low connectivity ahead, I want to download the latest 3 episodes from my favorite podcasts, so I can listen during my commute without buffering."
- "As a frequent traveler, when I'm at the airport with weak Wi-Fi, I want to confirm what's already downloaded, so I can plan whether to wait or board."

Aim for 5–10 user stories covering the main flows. They drive the design and inform QA.

Section 8: Functional Requirements

This is where you describe what the feature must do. Group by user story or by capability.

Example:

DOWNLOAD MANAGEMENT

- User can tap a "Download" button on any episode page.
- User can tap "Download" on a podcast page to download the latest 5 episodes (configurable in settings).
- User can see download progress on each episode (in-progress, completed, failed).
- User can cancel an in-progress download.
- User can delete a downloaded episode (manually).

OFFLINE PLAYBACK

- When the device is offline, the app must clearly indicate offline mode.
- Downloaded episodes must play without internet connection.
- The app must continue playback if connectivity drops mid-stream and the episode is downloaded.

STORAGE MANAGEMENT

- User can see total storage used by downloads.
- User can set a storage cap (default: 2GB).
- When near cap, the app prompts user to free space (suggesting oldest downloads to remove).
- Downloaded episodes are automatically deleted 30 days after the user has fully listened (configurable).

ETC...

Each requirement should be specific enough that an engineer can build it without further interpretation. If you find yourself writing "the app should be intuitive" — stop. That's not a requirement. Replace it with specific behaviors.

Section 9: UX & Design

This section links out to Figma (or your design tool) and describes the high-level flow at a sketch level. Include:

- Link to the Figma file (and a specific frame for each user story).
- Key screens and what they show.
- Notable interaction patterns.

Don't try to replicate the design in the PRD — Figma does that. The PRD should reference and complement, not duplicate.

Section 10: Edge Cases & Error States

This is where junior PMs lose points and senior PMs shine. List the edge cases you've thought about:

- What happens if the download fails midway?
- What happens if the device runs out of storage during download?
- What happens if the user signs out?
- What happens if the podcast is deleted from the catalog while downloaded?
- What happens if the user tries to download an episode they already have?
- What happens if the user is on a metered cellular connection?
- What happens if the download was paid premium content and the user cancels their subscription?
- What happens during background downloads with the screen off?

- What about a podcast in 25 languages — same logic for all?

Engineering will love you. This section also reveals your discovery rigor.

Section 11: Technical Considerations / Dependencies

Don't write the technical design — that's engineering's job. But note:

- Which existing systems will this affect? (Payment, content delivery, analytics, recommendation systems.)
- What dependencies exist? (Third-party SDKs, internal teams that own related systems.)
- What technical constraints have been discussed? (Size limits, expiry rules.)
- What technical risks has eng flagged?

This shows you've talked to engineering and aren't writing in a vacuum.

Section 12: Risks & Open Questions

Be explicit about what we don't know.

RISKS:

- Storage on low-end Android phones may be a barrier (40% of our users have <16GB free).
Mitigation: Configurable size cap; suggest deleting other content.
- Music license terms may not allow offline storage for all licensed content.
Mitigation: Confirmed with legal — only podcasts (which we own license for offline) qualify.
- Increased CDN cost from "download then delete" pattern.
Mitigation: Cache aggressively; CDN team estimates 12% cost increase, acceptable.

OPEN QUESTIONS:

- Should offline content count toward listening minutes for premium tier billing? (Need answer from finance.)
- Should we expire downloads at 30 days for free users vs unlimited for paid? (Pricing/package conversation.)
- What's the i18n approach for download UI on RTL languages?

These are signals to leadership and your team about where decisions are still pending.

Section 13: Launch Plan

How will this roll out?

- Week 1–6: Build.
- Week 7: Internal dogfood (employees only).
- Week 8: 1% rollout to iOS users in India only. Monitor crash rate, download success rate, storage complaints.
- Week 9: 10% if metrics healthy.
- Week 10: 50%.
- Week 11: 100% globally.

ROLLBACK CRITERIA:

- Crash rate increases by >0.5%.
- Download failure rate >5%.
- Disk-full support tickets exceed 20/week.

A great PRD names rollback criteria *before* launch. This prevents rationalization later.

Section 14: Out of Scope

What did we explicitly choose not to build?

- Wifi-only download mode (will reconsider in v2 based on data usage feedback).
- Auto-download for new episodes (deferred to v2).
- Downloading shared playlists (deferred).
- Web client offline reading (different team owns).

Out of Scope is one of the most important sections to manage stakeholder expectations.

Section 15: References & Appendix

Link to:

- Original research deck.
- Customer interview synthesis.
- Analytics queries that motivated this.
- Competitive analysis.
- Prior PRDs of related features.

The PRD becomes a node in a knowledge graph of decisions. Future PMs will thank you.

6.3 A Complete PRD Example

Below is a full PRD you can model your own after. This is for a hypothetical Indian fintech startup launching a "Round-Up Savings" feature.

PRD: Round-Up Savings (v1) — "SmartSave"

Author: A. Sharma, PM | Reviewers: T. Kumar (Eng Lead), R. Iyer (Design), M. Joshi (Compliance) Last updated: April 2025 | Status: Approved for build

1. TL;DR

We are launching SmartSave, a Round-Up Savings feature that automatically saves the spare change from a user's UPI/card transactions into a savings account or mutual fund. For every transaction, we round up to the nearest ₹10, ₹50, or ₹100 (user-configurable) and transfer the difference to their saving destination. Based on user research with 24 Indian millennials (HHI ₹6-18L), 78% reported "wanting to save but never getting around to it." We hypothesize that automating this behavior in the background of normal spending will lift our active-savings users from 18% of total user base to 28% within 6 months post-launch.

2. Background & Context

Our product currently provides UPI payments, bill pay, and basic mutual-fund investing. While 41% of users have linked their bank accounts and use payments daily, only 18% have ever made an investment. Discovery interviews (linked: [research-doc]) revealed that the dominant barrier is not knowledge or interest — it is *initiation*. Users say things like "I keep meaning to save," "I'll start next month," "I don't know how much to start with." This is a textbook job-to-be-done where the user wants to save *without thinking about it*. Round-up products (Acorns in the US, Moneyfarm in EU) have demonstrated this behavior change at scale.

This fits our 2025 strategic theme of "Convert payment users to investing users." We believe SmartSave is the highest-leverage way to bridge the two behaviors with minimal user activation friction.

3. Problem Statement

Indian millennials (HHI ₹6–18L) on our platform currently use payment features daily but rarely save or invest, despite stating they want to. They report inertia ("never get around to it") and uncertainty ("don't know how to start") in research. We know 78% of 24 interviewees described this exact dynamic. They have tried setting reminders, manual SIPs, or talking to advisors — but those approaches failed because they require active decision-making at recurring intervals. Solving this would mean savings starts happening passively in the background of their normal spending, measured by % of users with $\geq$ ₹500/month auto-saved.

4. Goals & Non-Goals

Goals	Non-Goals
Increase % of users with active savings flow from 18% → 28% in 6 months	Increase total mutual fund AUM in v1 (long-term goal, not v1)
Establish a recurring, automated savings habit	Provide investment advice or active recommendation
Round up UPI and debit card transactions automatically	Round up credit card purchases (deferred — different settlement)
Allow user to choose savings destination (bank, MF)	Provide tax-advantaged savings (NPS, ELSS) in v1
Indian users only	International expansion
Android + iOS	Web (separate roadmap)

5. Target Users

- *Primary*: Indian users aged 24–38, HHI ₹6–18L, who have already made at least one UPI payment in the last 30 days. We estimate this is ~280K WAU on our platform today.
- *Secondary*: Users who have linked bank accounts but haven't yet invested.
- *Out of focus*: Inactive users (no transaction in 30 days), enterprise-account users, users below KYC verification.

6. Success Metrics

Metric	Baseline	Target (6 months)	Source
Primary: % of WAU with active SmartSave	0%	28% (i.e., 28% of WAU enabled and saving $\geq$ ₹500/month)	App analytics
Secondary: Avg monthly auto-saved per user	n/a	₹650	Wallet ledger
Secondary: Conversion: SmartSave user $\rightarrow$ first MF investment	n/a	8% in 90 days	Cross-event analysis
Secondary: D90 retention of SmartSave users	n/a	+5pp vs non-SmartSave WAU	Cohort analysis
Guardrail: Customer complaints / support tickets	baseline	< 1.5 $\times$ baseline	Zendesk
Guardrail: Failed round-up transactions	n/a	< 0.5%	Eng telemetry
Guardrail: Aggregate fund flow into MF (we don't push uninformed)	baseline	No change unless explicit user opt-in	Compliance reports

If we do not see $\geq 12\%$ adoption among targeted users at 8 weeks post-launch, we will pause new feature development and run targeted user research to understand barriers.

7. User Stories

- *As a daily UPI user paying ₹47 for chai, I want the app to round up my transaction to ₹50 and save ₹3 automatically, so I can build savings habits without friction.*
- *As a new user activating SmartSave, I want to choose my round-up amount (₹10/₹50/₹100) and saving destination, so I can match the feature to my comfort level.*
- *As a user about to spend a lot on a wedding, I want to pause SmartSave temporarily, so I can avoid double-deductions on planned big spends.*
- *As a worried user, I want to see exactly how much was saved this month and where it went, so I can trust the automation.*
- *As a user discovering the feature, I want a simple onboarding flow that shows me the math, so I can understand it without confusion.*

8. Functional Requirements

Activation flow:

- New SmartSave entry point in the home screen, directly under "UPI Payments."
- Activation flow: explanation $\rightarrow$ choose round-up amount $\rightarrow$ choose destination $\rightarrow$ review and confirm.
- Default round-up: ₹10. Defaults are research-tested.
- Destination options: linked savings account (default), liquid debt mutual fund, gold (deferred).

Round-up logic:

- For every successful UPI/debit card transaction by user, calculate $\text{roundUp} = \text{ceil}(\text{transaction} / \text{increment}) \times \text{increment} - \text{transaction}$.
- Aggregate round-ups daily.
- Once daily aggregate $\geq ₹50$ (or end of week, whichever comes first), execute transfer.
- Transfer must succeed; on failure, retry up to 3× with exponential backoff. Notify user if all fail.

Visibility:

- "SmartSave this month" tile on home screen showing total saved and trend.
- Detailed transaction-by-transaction history.
- Monthly summary statement (email + in-app).

Pause / cancel:

- One-tap pause from home screen tile.
- Resume requires explicit user action.
- Cancel requires confirmation modal explaining the impact.

[Detailed flow diagrams in Figma: [link](#)]

9. UX & Design

Figma file: [\[link to file\]](#) Key flows:

- Activation: [Frame 1.1]
- Daily home screen tile: [Frame 1.2]
- Round-up history: [Frame 1.3]
- Pause confirmation: [Frame 1.4]

Design principles:

- Show users the math. Trust requires visibility.
- Default to safer choice (lower round-up amount).
- One screen for activation; do not stretch to 4 screens.

10. Edge Cases & Error States

- *Insufficient balance for round-up*: Skip that transaction's round-up and notify user via in-app banner. No retry.
- *Failed transfer*: Retry 3×, then notify user. Money stays in user's main account.
- *User changes destination mid-month*: Apply new destination from next round-up; previously transferred amount stays where it was sent.
- *Transaction reversal/refund*: If a UPI transaction is reversed within 7 days and round-up was already transferred, do not auto-reverse the round-up (user would have lost money).

- *Multiple pending round-ups during outage:* Queue and process when service restored; user sees a single summary, not a flood of transfers.
- *User KYC lapses:* Pause feature; notify user to refresh KYC.
- *Account closure:* Immediately stop round-ups; transfer pending aggregate to user's primary account.

11. Technical Considerations / Dependencies

- Depends on: Payments service (transaction event stream), Wallet service (transfer execution), Notification service.
- New: Round-up calculation worker (eng team estimates 4 weeks).
- Compliance: Confirmed by M. Joshi that round-up automation falls under existing user authorization on linked savings; explicit consent UI required.
- Engineering risk: Transaction event stream has 0.3% out-of-order delivery rate; must handle.

12. Risks & Open Questions

Risks:

- (R1) **Adoption risk.** Round-up features have low awareness in India. We may see <10% adoption. Mitigation: Onboarding banner + email campaign + in-app discovery.
- (R2) **Trust risk.** First time users feel money is being "taken" without consent. Mitigation: Clear confirmation flow, daily summary, easy pause.
- (R3) **Cost risk.** High volume of small mutual fund transactions could exceed our payment-rail SLAs. Mitigation: Aggregate daily, transfer in batches.
- (R4) **Regulatory risk.** SEBI / RBI review of "automatic micro-investing" — flagged with legal. No restriction expected based on current rules.

Open questions:

- Q1: Should we enable round-up for credit card transactions in v2?
- Q2: Should we offer a yearly summary as a CTA to set a savings target?
- Q3: Should we tier the feature (e.g., advanced rules in premium)?

13. Launch Plan

- **Week 1-6:** Build.
- **Week 7:** Internal employee testing (50 employees with linked accounts).
- **Week 8:** 1% A/B rollout (Mumbai + Bangalore users only).
- **Week 9-10:** 10% rollout if metrics healthy.
- **Week 11:** 50%.
- **Week 12:** 100% nationwide.

Rollback criteria:

- Customer complaints exceed 2× baseline.
- Failed transfers exceed 1%.
- Crash rate up by >0.3%.
- Negative coverage in major financial press.

Go-to-market:

- Coordinated launch email to all eligible users.
- In-app feature discovery banner.
- Influencer partnerships with Indian personal-finance YouTubers (CA Rachana Ranade, Pranjal Kamra) — coordinated with marketing.
- Press release through PR team.

14. Out of Scope

- Credit card round-ups (v2)
- Gold as a destination (v2)
- Custom round-up rules (e.g., "only round up restaurant spends")
- Joint accounts / family round-ups
- Pre-paid wallet round-ups

15. References & Appendix

- Customer interview synthesis: [\[link\]](#)
- Competitive analysis (Acorns, Jar, Niyo): [\[link\]](#)
- Engineering architecture doc: [\[link\]](#)
- Compliance memo: [\[link\]](#)
- Pricing & profitability model: [\[link\]](#)

End of example PRD.

This is roughly 1,000 words long — typical for a focused feature. A platform initiative or major new product can be 3,000–5,000 words. The point is *fit-for-purpose*: enough to align everyone, no more.

6.4 Bad PRD vs Good PRD — Side-by-Side

A common pattern: junior PMs write PRDs that are mostly feature lists with no problem framing, no metrics, and no edge cases. Let's compare.

Bad PRD excerpt:

"Add a notifications feature so users get notified about new content. The notifications should be configurable. There should be a settings page where users can turn them on or off. Notifications should look good and feel native."

What's wrong:

- No problem statement. Why are we building this?
- No metrics. How will we know it worked?
- "Look good and feel native" is not a requirement; it's a wish.
- No edge cases (silent mode? do not disturb? international? etc.).
- No user stories. Who uses this and how?

Good PRD excerpt:

Problem: New podcast episodes from a user's followed shows go unnoticed for an average of 5.2 days, leading to 41% of episodes never being played. Among users who *do* enable browser notifications, average time-to-first-play drops to 1.4 days and 88% of episodes are played. We hypothesize that proactive notifications, with smart defaults, will lift episode-play rate from 59% → 75% across the user base.

Primary metric: Episode play rate within 7 days of release, measured by cohorts of "released this week" episodes; baseline 59%, target 75% within 12 weeks.

Guardrail: Notification opt-out rate must stay <8%. If users reject notifications en masse, we've failed at trust.

User story 1: As a user with 6 podcast subscriptions, when a new episode is released, I want a single, batched notification (not 6 separate ones) at a time I'm likely to listen, so I can choose what to play without overwhelm.

This second version has clarity, hypothesis, measurable goals, guardrails, and user-story specificity.

6.5 User Stories vs Requirements vs Outcomes

Three terms PMs often blur. Let's separate.

	Definition	Example
Outcome	The user/business goal we're optimizing for	"Increase episode plays in week 1"
User story	A specific user journey through the product	"As a commuter, when a new episode is released..."
Requirement	A specific behavior the product must exhibit	"The system must batch notifications when 3+ are pending within 30 minutes"

The relationships:

- **Outcomes** drive what we build.
- **User stories** translate outcomes into specific user contexts.
- **Requirements** translate user stories into specific system behaviors.

A great PRD has all three layers, in this order.

A common failure mode: writing requirements without user stories. The team builds correctly per requirements but solves the wrong user need.

Another common failure: writing user stories without outcomes. The team builds nice flows but no one knows whether it actually worked.

6.6 Working with Engineering — The Most Important Relationship

Your relationship with the engineering team determines whether you ship. This is not optional learning; it is core PM craft.

Principles

1. Engineers don't work for you. They work *with* you.

The PM-engineer relationship is a partnership. Engineers have their own incentives (technical quality, sustainable systems, good architecture, learning). Treating them as ticket-takers destroys trust permanently.

2. Earn the right to give input on technical questions.

You will be tempted to second-guess technical estimates, push back on architecture decisions, or suggest "easy" alternatives. *Don't* — until you've built sufficient credibility. Most "easy alternatives" PMs propose are actually harder once you understand the system. Ask, learn, then suggest.

3. Share context generously.

Engineers care more when they understand *why*. Walk them through the user research. Show them the customer support tickets. Get them on customer calls. The investment in context-sharing pays off in better technical decisions, more proactive bug catches, and fewer "is this really worth building?" debates.

4. Defend the team's time and quality.

When sales pushes you to commit to a 2-week deadline that the team estimated at 5 weeks, your job is to *defend the team's estimate*, not to relay sales pressure. Becoming the team's filter against external pressure is what earns lasting trust.

5. Be reliable.

Show up on time. Reply to questions same-day. Don't change scope mid-sprint without conversation. Don't schedule recurring meetings you don't actually use. Reliability compounds.

Practical Practices

Daily / Sprint:

- Attend standups but don't dominate them. Standups are for the team, not for status reporting to you.
- Be available for spec questions during the day. Engineers can't ship if they're blocked on PM clarifications.
- Refactor PRDs as you learn. A PRD is a living doc, not a contract.

Weekly:

- 1:1 with the engineering manager / tech lead. Discuss what's going well, what's blocking, what you're seeing in user data.
- Quick demo of in-progress work. Catch misalignment early.

Per project:

- Kickoff: PRD review with the whole team. Make space for engineers to push back, propose alternatives, ask questions.
- Mid-project check-in: Are we still building toward the same goal? Has anything changed?
- Pre-launch: Walk through the full feature together. Simulate user flows. Find bugs.
- Post-launch retro: What worked, what didn't, what would we do differently?

Communicating with Engineering

Don't:

- Say "this is easy."
- Give arbitrary deadlines without involving the team.
- Argue about architecture publicly.
- Say "the customer wants X" without explaining the underlying problem.
- Hide bad news until it explodes.

Do:

- Say "I don't know how complex this is — can we scope together?"
- Share trade-offs and let the team weigh in.
- Defer to engineering on technical implementation; insist on user requirements.
- Explain *why* a request matters with research/data context.
- Surface risks early; bad news travels faster than good news.

A Common Conflict and How to Handle It

Scenario: You scoped a feature for 4 weeks. Engineers say it's actually 7 weeks because of a complication you didn't foresee.

Bad response: "We committed to 4 weeks. You need to figure out how to make it work."

Good response: "Walk me through what changed. Is it that the original scope was incomplete, or did we hit unexpected complexity? Let's look at the original spec together — is there scope we can defer to a v2 to hit a tighter timeline? Or should we accept the 7 weeks and re-communicate with stakeholders?"

The first kills trust. The second deepens it. You may still ship later than planned, but you'll ship with a team that wants to ship for you next time.

6.7 Working with Design

The PM-designer relationship is similar to PM-engineering, with its own nuances.

Principles

1. Don't design in PRDs.

State the user problem, the user stories, the constraints, and let designers explore the solution space. If you've already designed it (or made specific UI decisions in the spec), you've eliminated the value the designer brings.

2. Review designs with empathy.

When you see a design and disagree, don't say "I don't like this." Ask: "Help me understand the choice here. I was expecting X — what made you go with Y?" Often the designer has solved a problem you didn't know existed.

3. Speak in terms of user outcomes, not aesthetic preferences.

Bad: "I'd put the button on the left." Good: "I'm worried that the most important action isn't visible enough for users in the [X scenario]. What if we tested moving it more central?"

4. Defend design quality.

When eng pushes for a faster, uglier solution, the PM should weigh the trade-off carefully. Sometimes speed wins. Sometimes design quality wins. The PM's job is to make this trade-off thoughtfully, not to default in either direction.

5. Get design involved early.

If you write the PRD without design input, the design phase becomes "translate the PRD into screens." If design is involved from problem framing, you'll get a much better product.

Working Practices

- Design sprints / kickoffs at the start of any major feature.

- Mid-design reviews to align on direction before too much fidelity is invested.
- Co-attendance at user research sessions. Designers learn from interviews directly; don't filter through PM notes.
- Joint launch reviews with the engineer + designer + PM seeing the working product together.

6.8 Working with Data Science / Analytics

Data scientists are your partners for measurement, experimentation, and analysis.

Principles

1. Bring questions, not requests.

Don't say "Pull me a report on retention." Say "I'm trying to understand whether users who do X retain better than users who don't. What would you suggest to investigate this?" The first is a clerk request; the second is a conversation between problem-solvers.

2. Trust their judgment on statistical questions.

If a data scientist says "this A/B test isn't significant," don't argue. If you don't understand why, ask them to teach you. Statistical mistakes by PMs cost companies millions in misguided decisions.

3. Co-design instrumentation.

Before any feature launches, walk through with data: what events are we firing? What dashboards will measure success? Are we logging enough to do segmentation later? Don't ship and then ask for analysis.

Anti-patterns

- Asking for data in random Slack messages instead of structured requests.
- Overruling statistical interpretations because you have a "feeling."
- Cherry-picking metrics that support what you wanted to do.
- Not engaging until launch, then panicking that the data isn't there.

6.9 Working with Business Functions (Sales, Marketing, Support, Legal, Finance)

These are your stakeholders. They aren't your team — they're your partners with their own incentives.

Sales

Sales wants to close deals. They will ask for custom features, accelerated timelines, exceptions to policy. Your job:

- Listen carefully to objections from real lost deals.
- Distinguish *patterns* (real product gaps) from *outliers* (one-off customizations).
- Be a data-informed advocate ("3 deals lost this quarter for X reason — let's discuss").
- Build a feedback channel (e.g., a regular sales-product sync, a shared "lost deal" Notion page).

Marketing

Marketing positions and promotes the product. They need:

- Clear messaging on what the product does and who it's for.
- Visibility on launches early (8 weeks ideally).
- Talking points, screenshots, demo flows.
- Beta access for content creation.

A great PM treats marketing as a force multiplier. A bad PM treats them as a launch-time afterthought.

Support / Customer Success

Support and CS hear customer pain in real-time. They are the most underused source of product insight in most companies. Build a regular rhythm:

- Weekly review of top 10 support categories.
- Quarterly review of CS-flagged churn patterns.
- Open Slack channel for support to flag emerging issues.

Legal & Compliance

Especially in regulated domains (fintech, healthtech, B2B with data privacy). Bring legal in early on any:

- New data collection.
- New monetization models.
- Cross-border data flows.
- AI/ML decisions affecting users.
- Pricing changes affecting existing contracts.

Late legal involvement is the #1 cause of preventable product launch delays.

Finance

Finance cares about unit economics, margins, and forecasts. They need:

- Clarity on cost implications of features (cloud spend, COGS).
- Pricing decisions discussed with their input.
- Forecasts that connect product launches to revenue impact.

6.10 Agile, Scrum, Kanban — What Actually Matters

The agile world has invented an enormous amount of jargon. Here's what actually matters as a PM.

The Core Insight of Agile

Software is unpredictable enough that long upfront plans fail. Better to work in short cycles, ship something usable, learn, and adapt. That's the entire idea of agile. Everything else is implementation detail.

The Agile Manifesto (2001) lists four values:

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

Note: "over" doesn't mean "instead of" — it means "more weight on the left side when in conflict." A PM who reads "working software over documentation" as "don't write PRDs" has misunderstood. PRDs exist to enable working software.

Scrum

Scrum is one specific implementation of agile, with prescribed roles, ceremonies, and artifacts.

Roles:

- **Product Owner:** Manages the backlog (often the PM, especially in smaller orgs).
- **Scrum Master:** Facilitates the process. Sometimes a separate role; sometimes the engineering manager.
- **Development Team:** Engineers, designers, QA — those building the product.

Ceremonies:

- **Sprint Planning** (start of sprint): What will we build this sprint? Why?
- **Daily Standup** (15 min): What I did yesterday, what I'll do today, blockers.
- **Sprint Review** (end of sprint): Demo what we built. Get feedback.
- **Retrospective** (end of sprint): What went well? What didn't? What will we change?

Artifacts:

- **Product Backlog:** Prioritized list of all things to build.
- **Sprint Backlog:** Subset committed to this sprint.
- **Increment:** Working software delivered each sprint.

Sprint duration: Usually 2 weeks. Sometimes 1 or 3.

Kanban

A different agile flavor, focused on continuous flow rather than time-boxed sprints. You have a board with columns (To Do, In Progress, Review, Done). Work flows through. Limit work-in-progress (WIP) to maintain focus.

Kanban suits:

- Operational/maintenance teams (support, on-call, infrastructure).
- Teams with unpredictable urgent work.
- Small teams that don't need ceremony overhead.

Hybrid Approaches

Most modern teams run a *hybrid*: Scrum-like sprint planning, but with Kanban-style continuous flow within sprints; lightweight ceremonies; flexible scope. The label matters less than the rhythm.

What Actually Matters (The PM's Filter)

For a PM, focus on these execution practices regardless of label:

1. **A clear, visible backlog**, sorted by priority.
2. **Short cycles (1-2 weeks)** with explicit goals.
3. **A demo or check-in** every cycle to see real progress.
4. **A retro** at least monthly to improve the team.
5. **Limited WIP** — too much in-progress kills throughput.
6. **Clear definitions of done** — when is something "shipped"?

If your team has these, you're doing agile. If not, no amount of ceremony fixes the underlying gap.

Anti-patterns

- **Story-point obsession.** Whether a task is "5 points" or "8 points" rarely matters. Time spent debating point estimates is time not spent building.
- **Standup as status report to PM.** Standup is for the team to coordinate. If you're using it to interrogate engineers, you've broken it.

- **Rigid sprints in a fluid context.** If your team is supporting on-call and getting fires constantly, sprint-based planning will feel artificial. Adapt.
- **Backlog hoarding.** Backlogs with 800 items aren't backlogs — they're graveyards. Periodically purge.
- **Estimating to satisfy stakeholders.** When a CEO asks "when will it ship?", an honest "we don't know yet — here's the milestones we're targeting" is better than a fake date.

6.11 Sprint Planning, Backlog Grooming, and Trade-offs

Sprint Planning Practical

Inputs to a sprint planning session:

- A groomed, prioritized backlog of 1.5–2× the sprint's capacity.
- Each top item has clear acceptance criteria.
- Designs are ready (or it's a research/spike sprint).

In the session:

1. Review the sprint goal (1 sentence — what are we trying to achieve?).
2. Review top backlog items — discuss complexity, dependencies, risks.
3. Engineering commits to a realistic scope based on team capacity.
4. PM accepts the scope or has a conversation about trade-offs.

Backlog Grooming Practical

Goal: keep the top of the backlog ready for the next 2 sprints.

Frequency: 1× per sprint, ~1 hour.

What to do:

- Top 20 items: ensure each has a clear problem, user story, acceptance criteria.
- Items beyond top 20: light review only.
- Old, stale items: kill mercilessly. If nobody's pulled it in 6 months, it's probably never going to happen.

Common Trade-offs in Sprint Execution

- **Eng wants to refactor a system** vs *PM wants to ship a feature*. Resolve by: explicit "tech debt budget" (e.g., 20% of every sprint allocated to tech debt of eng's choice).
- **Designer wants polish iteration** vs *PM wants to ship MVP*. Resolve by: shipping V1, planning V1.5 polish iteration explicitly.

- **Sales wants a customer's custom feature** vs *PM has prioritized other work*. Resolve by: cost-of-delay calculation, transparency to customer, sometimes a paid customization path.

6.12 Release Planning and Delivery Risks

Release Types

- **Alpha:** Internal-only. Test with employees.
- **Beta / Early Access:** Limited external. Selected users opt in or are chosen.
- **Soft launch:** Released to a subset (e.g., one country, or 5% of users) without marketing.
- **General availability (GA):** Available to all users. Often marketed.

Use the lightest release that gives you the data and validation you need. Don't go straight to GA on a major change.

Feature Flags

The single most important release tool for modern PMs. A feature flag is a server-side switch that lets you turn a feature on or off for specific users without re-deploying code. Modern setups (LaunchDarkly, Optimizely, GrowthBook) make this seamless.

Why feature flags matter:

- **Gradual rollout.** Start at 1%, watch metrics, ramp up.
- **Instant rollback.** If a bug appears at 50% rollout, flip the flag, restored in seconds.
- **A/B testing native.** Same infrastructure that controls rollout enables experimentation.
- **Deploy / launch decoupling.** Engineers can deploy code anytime; PMs choose when to "launch" by flipping the flag.

If your team isn't using feature flags for non-trivial features, it's a process gap to fix.

Common Delivery Risks

1. **Dependency risk.** Your feature depends on another team's API; that team is delayed.
Mitigation: Identify dependencies in PRD; track external commitments weekly; have a fallback plan (mock data, degraded mode).
2. **Scope creep risk.** "We're already building X; can we just add Y?" *Mitigation:* Strong Out-of-Scope section; explicit conversation when scope changes.
3. **Quality risk.** Bug count too high near launch date; team pushes to "fix in next sprint."
Mitigation: Define quality bar in PRD; if not hit, slip launch (preserve trust > preserve date).
4. **Underspecified edge cases.** "We didn't think about [X scenario], engineering doesn't know what to do." *Mitigation:* Dedicated edge cases section; midpoint review with eng to surface unknowns.

5. **Adoption risk.** Feature ships, no one uses it. *Mitigation:* Pre-launch GTM plan; in-app discovery; user research on activation.

6. **Team energy risk.** Long projects burn teams out. *Mitigation:* Break long projects into milestones with shipping moments; celebrate wins; reduce scope rather than push harder.

The Pre-Launch Checklist

A practical pre-launch checklist:

PRODUCT READINESS:

- ☐ All acceptance criteria met
- ☐ Edge cases tested
- ☐ Mobile (iOS, Android) parity verified
- ☐ Localization complete for required markets
- ☐ Accessibility verified (basic WCAG)
- ☐ Performance acceptable on low-end devices

INSTRUMENTATION:

- ☐ All success metrics instrumented
- ☐ Dashboards ready and tested
- ☐ Funnel events firing correctly
- ☐ Crash and error tracking configured

ROLLOUT:

- ☐ Feature flag configured
- ☐ Rollout plan documented (1% → 10% → 50% → 100%)
- ☐ Rollback procedure tested
- ☐ Engineering on-call rotation confirmed

GO-TO-MARKET:

- ☐ Marketing copy approved
- ☐ Press release drafted (if applicable)
- ☐ Sales enablement done
- ☐ Support team trained
- ☐ Help center articles published
- ☐ In-app announcement scheduled

LEGAL / COMPLIANCE:

- ☐ Privacy review complete
- ☐ Terms of service updated if needed
- ☐ Compliance sign-off (where required)

BUSINESS:

- ☐ Pricing/packaging decision finalized
- ☐ Internal stakeholder communication ready
- ☐ Success metrics + review cadence defined
- ☐ Post-launch retrospective scheduled

Use it. Adapt it to your context. The 30 minutes spent reviewing this list saves days of post-launch fires.

6.13 Comprehensive Practice Exercises

Exercise 6.1 — Write a Full PRD

Choose one of these problems and write a full PRD using the template in 6.2:

1. AI-powered "explain like I'm 5" feature for a financial education app for Indian first-time investors.
2. Feature to split bills automatically among friends in a UPI payments app.
3. Offline mode for a video-streaming app for users in low-connectivity rural India.
4. AI summarization of long Slack threads.

Aim for 800–1,500 words. Use all 15 sections.

Exercise 6.2 — Diagnose a Bad PRD

Below is a (fictional) PRD excerpt. Identify at least 8 things wrong with it.

PRD: Better Search

Background: Search is bad. Users have complained.

Goal: Make search better.

Requirements:

- Add filters
- Make it faster
- Show better results
- Should be intuitive
- Mobile-friendly

Timeline: 6 weeks.

Success: Users will be happier.

Exercise 6.3 — Write User Stories

For a hypothetical feature ("send-money-to-multiple-friends in a UPI app"), write 5 user stories using the format from Section 6.5. Cover at least these scenarios: first-time use, frequent power user, error case, privacy-sensitive case, multi-currency.

Exercise 6.4 — Edge Case Brainstorm

You're shipping an "auto-renew subscription" feature. Brainstorm at least 15 edge cases. Categorize them as: critical (must handle in v1), important (handle in v1.1), or nice-to-have.

Exercise 6.5 — Pre-launch Readiness Audit

Imagine you are 2 weeks from launching the SmartSave feature in section 6.3. Conduct a written pre-launch audit using the checklist in 6.12. Identify 3 likely gaps and how you'd close them.

Exercise 6.6 — Stakeholder Conflict Resolution

Write your response to this email from the Head of Sales:

"Hi [you], I just lost a 200-seat deal because we don't have audit log export. Their security team wouldn't approve us without it. The customer says they'd sign next week if we had this. Can you ship it next sprint? It's a make-or-break for the quarter and I've put my number behind it."

Solutions

Solution 6.2 — Bad PRD diagnosis:

The fictional "Better Search" PRD has at least these issues:

1. **No problem statement.** "Search is bad" is not a problem definition. Bad how? For whom? How often? What evidence?
2. **No user segment.** Who is affected? All users equally? Mobile users? Power users? New users?
3. **No success metric.** "Users will be happier" is not measurable. We need a baseline and target — e.g., "search-to-click rate from 32% → 45%."
4. **Vague requirements.** "Make it faster" is not a requirement. Faster how? From what to what?
5. **No user stories.** No description of actual user journeys.
6. **No edge cases.** What happens with 0 results? Long queries? Misspellings? Multilingual?
7. **No design references.** Where are the wireframes?
8. **No technical considerations.** Search architecture matters; what's the underlying engine?
9. **No risks or open questions.**
10. **Arbitrary timeline.** "6 weeks" without team estimation.
11. **No launch plan.** How do we roll this out? A/B vs full?
12. **No out-of-scope.** What aren't we doing?
13. **No reference to research or data.** Is this based on user research, support tickets, or just opinion?

This is the kind of PRD that produces a 3-month engineering project resulting in users still being unhappy.

Solution 6.6 — Sales conflict response:

Hi [Sales Lead],

Sorry to hear about the lost deal — that's painful. Audit log export is a real gap and worth a serious look. Before I commit, let me get specific so we make the right call.

Quick clarifying questions:

1. What did the customer specifically need — full audit log of admin actions, user actions, both? Export format (CSV, JSON, SIEM-compatible)? Frequency (real-time stream vs daily batch)?
2. Is this a pattern? How many other deals (recent + pipeline) have flagged audit logs as a blocker?
3. Their timeline of "next week" — is there flexibility for a 4-6 week delivery if we commit to a specific date?

My initial read: a real audit log feature with proper retention, scoping, and export typically takes 3-5 weeks of focused engineering work. Doing it in one sprint would either ship a low-quality version (loss of trust if their security team rejects it) or pull eng off the activation initiative we committed to leadership last month.

Here's what I propose:

- I'll talk to 3 sales reps + your top 5 enterprise prospects this week to understand the audit-log requirement deeply. If it's a pattern (which I suspect), we'll prioritize it for the Q3 cycle starting in 4 weeks.
- For this specific lost deal: can we re-engage with a "we have this on our roadmap, deliverable in [date]" commitment? This often works for security-conscious enterprise buyers if the timeline is reasonable.
- If it's truly make-or-break and a pattern across 3+ deals, I'll come to leadership and we'll decide whether to insert it now (accepting that activation work slips).

Can we sync at 4pm today to align? I want to make sure we're solving the right problem and that we're honest with the customer.

[You]

This response: (a) takes the request seriously, (b) asks for specifics, (c) names trade-offs honestly, (d) offers a path forward, (e) escalates appropriately. It doesn't reflexively comply, doesn't reflexively deny, and protects both customer and team.

6.14 AI Leverage in Execution

Use case 1: PRD drafting assistant.

Prompt: "I want to write a PRD for [feature]. Here's the user problem [paste]. Help me draft a structured PRD using the 15-section template I'm pasting below. For each section, generate a strong first draft based on the problem. Mark sections that need my expert input or specific data."

Use AI to accelerate the boring 60% of writing. You sharpen the strategic 40%.

Use case 2: Edge case brainstorming.

Prompt: "For this feature [paste functional requirements], generate 25 edge cases or error states a user might encounter. Categorize by: input edge cases, system error cases, network/connectivity cases, security cases, and rare-but-critical cases. Note ones that look low-likelihood but high-impact."

This single prompt catches more bugs than weeks of QA in some cases.

Use case 3: Stakeholder communication drafting.

Prompt: "I need to push back on a feature request from [stakeholder type]. The request is [paste]. The reasons not to do it now are [paste]. Help me draft a response that acknowledges the request, explains my reasoning with empathy, and offers a path forward."

Use case 4: User story expansion.

Prompt: "Here are my 3 main user stories [paste]. Generate 8 additional user stories that I might be missing — covering: first-time use, power users, error scenarios, edge users (accessibility, low-end devices, slow connections), and adversarial users (those who'll try to game or break the feature)."

Use case 5: PRD critique.

Prompt: "Paste my PRD here: [paste]. Critique it harshly as a senior PM at Google would. What's missing? What's vague? Where am I solutioning instead of problem-defining? Where are my metrics weak? What edge cases am I missing? Mark each comment with severity (critical/major/minor)."

Use case 6: Engineering / sales / design dialogue rehearsal.

Prompt: "I have a meeting tomorrow with engineering about a scope debate. They estimate 6 weeks; I think 4 is possible. Roleplay as the senior engineer — push back realistically. After we talk, give me feedback on whether my reasoning holds up."

Use case 7: Release plan drafting.

Prompt: "For the feature in this PRD [paste], propose a phased rollout plan. Cover: alpha, beta, soft launch, GA. For each phase: who's the audience, what success criteria advance us, what failure criteria roll us back, and how long should each phase last."

6.15 References for Part 6

Books:

- *Inspired* — Marty Cagan. Foundational on PM-engineer-design collaboration. ([Amazon](#))
- *Empowered* — Cagan & Jones. On building empowered teams. ([Amazon](#))
- *Shape Up* — Ryan Singer (Basecamp). Free online; alternative to sprint-based agile that many top teams admire. (basecamp.com/shapeup)
- *The Phoenix Project* — Gene Kim. A novel about DevOps; PMs gain deep insight into engineering reality. ([Amazon](#))
- *Accelerate* — Forsgren, Humble, Kim. The science of high-performing engineering organizations. ([Amazon](#))
- *Sprint* — Jake Knapp. Google Ventures' design sprint methodology. ([Amazon](#))
- *User Story Mapping* — Jeff Patton. The craft of user stories. ([Amazon](#))

Essays / Articles:

- "Good PRDs and Bad PRDs" — many sources; search for Cagan's writing on this. ([SVPG](#))
- "Writing PRDs" — Lenny's Newsletter has multiple deep-dives. (lennysnewsletter.com)
- "The Anatomy of a Great PRD" — First Round Review. (review.firstround.com)
- "Working Backwards: Writing the Press Release First" — Amazon's PR/FAQ method. (Search: "Amazon PR FAQ.")
- "PM-Engineer Conflict, Resolved" — Will Larson's writing on PM-eng collaboration. (lethain.com)
- "What does a Product Manager actually do?" — multiple Shreyas Doshi threads. ([@shreyas](#))

Tools:

- **PRD writing/collaboration:** Notion, Coda, Confluence, Google Docs.
- **Issue tracking:** Linear (modern, beloved), Jira (enterprise standard), Shortcut, Pivotal Tracker.
- **Design tools:** Figma (industry standard), Sketch.
- **Feature flags:** LaunchDarkly, GrowthBook (open-source), Optimizely, Statsig.
- **Roadmap tools:** Productboard, Aha!, ProductPlan.
- **Sprint planning:** Jira, Linear's cycles, GitHub Projects.

Templates (free):

- Lenny's PRD template — widely used. (lennysnewsletter.com)
- Notion's PRD templates — multiple available.
- Amazon's "Working Backwards" template — multiple online sources.
- Marty Cagan's product brief — on SVPG.

Practice resources:

- Write a PRD per week for a feature you'd like to see in any product. Get peer feedback.
 - Read your own past PRDs from 6 months ago. Notice what you'd write differently now.
 - Interview engineers about their best and worst PRDs ever. Learn from real reactions.
-

End of Part 6. Up next: how to measure whether what you built actually worked. -e

\newpage